

TITLE OF THE PROJECT

E-commerce Order Fulfillment Automation Backend

NAME OF SUPERVISOR

ANIKA FARZANA

Junior Lecturer,

Department of Computer Science and Engineering

LIST OF TEAM MEMBERS

Name	ID
Saika Sarara	230042159
Adiba Ahsan Raiyan	230042161
Maliha Tasnim Khan	230042127

PROJECT OVERVIEW

The **E-commerce Order Fulfillment Automation Backend** is a **business-side console system** that automates the internal

order-processing workflow of an e-commerce company. Beginning *after* a customer places an order, the system automates:

- order validation
- stock verification
- invoice generation
- shipping simulation
- detailed workflow logging

Built in **Java (CLI)** with **file-based storage**, the design is modular and OOP-driven, modeling how real fulfillment pipelines operate behind the scenes.

MOTIVATION BEHIND THE PROJECT

During the festival season, we noticed a small online shop in Bangladesh repeatedly getting our orders wrong. Without any automation, their team mismatched stock, missed invoices, and shipped late. As customers it was frustrating—and it made clear that the issue wasn't the products, it was the workflow.

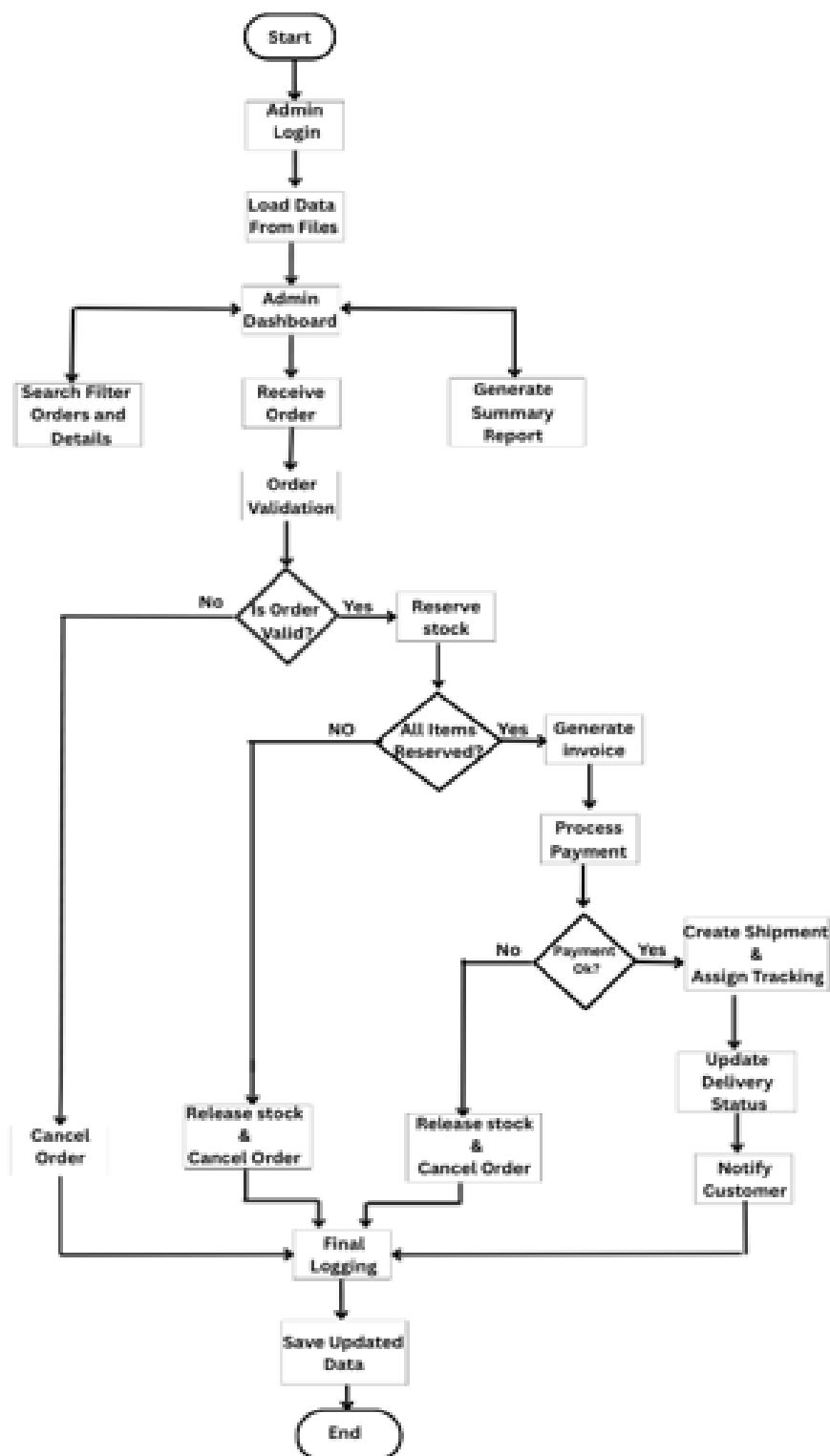
That's where the **E-commerce Order Fulfillment Automation Backend** began.

Instead of building another customer interface, we set out to recreate the *backstage* of an order: validation, stock checks, invoicing, and shipping. It's our way of turning the invisible journey of an order into something we can design, automate, and understand—through pure logic and clean, well-structured workflow steps.

FLOW CHART/CLASS DIAGRAM

The flowchart will represent the most accurate structure based on the current understanding, covering:

- Classes for Admin, Order, Item, Invoice, Inventory, PaymentProcessing, ShippingSimulation, Log, Report, Workflow, DataPersistence
- Relationships between modules
- Order flow from login to order validation, stock reservation, payment processing, shipping, and final logging.



KEY FEATURES (USE-CASE DESCRIPTIONS)

1. Secure Admin Login

- Admin-only access to the system
 - Passwords are stored using **hashed values**, not plain text
 - Prevents unauthorized access to backend operations
-

2. Order Intake

- Orders are loaded from structured JSON files
- Required fields: `orderId`, `items[ ]`, `address`, `paymentMode`
- Orders are queued for processing

3. Order Validation

- Checks for valid order structure
- Ensures non-empty items and positive quantities
- Invalid orders are cancelled and logged with reasons

4. Inventory Verification & Reservation

- Verifies stock availability for each item
- Uses a **whole-order rejection policy** if any item is unavailable
- On failure: stock is released and order is cancelled
- On success: stock is reserved

5. Billing & Payment Processing

- Calculates total cost from item prices and quantities

- Generates invoice with unique ID (**INV-YYYYMM-####**)
 - Simulated payment handling:
 - **COD** (auto-approved)
 - **MockCard** (approve/decline simulation)
 - Payment failure triggers stock rollback and cancellation
-

6. Shipping Simulation

- Shipment created with unique tracking ID (**TRK-#####**)
 - Delivery status progresses step-by-step:
 - **PACKED → SHIPPED → OUT_FOR_DELIVERY → DELIVERED**
 - Admin controls status updates
-

7. Admin Dashboard

- Menu-driven CLI dashboard
 - View all orders with status and summary
 - Search and filter orders by ID or status
 - Access order details, invoices, tracking, and logs
-

8. Manual Status Control

- Admin can manually update shipment status
 - System prevents invalid state transitions
 - Every update is logged for audit purposes
-

9. Workflow Logging & Audit Trail

- Each workflow step is logged with:
 - timestamp
 - orderId

- step name
 - outcome and message
 - Logs stored per order and in a master log file
-

10. Reporting System

- Automatically generates summary reports showing:
 - total orders processed
 - successful vs cancelled orders
 - cancellation reasons
 - total revenue from completed orders
 - Reports stored as files for review
-

11. Data Persistence

- Uses file-based storage:

- `products.json`
 - `orders.json`
 - `customers.json`
 - `invoices.json`
 - Data is serialized when saved and deserialized when loaded
 - Ensures data remains available across program runs
-

12. Test Scenarios

- Includes seeded test data
- Covers:
 - successful order flow
 - invalid order
 - stock shortage

- payment failure

TOOLS AND TECHNOLOGY

- **Programming Language:** Java
- **Execution Environment:** Console-based (CLI) application
- **Version Control:** Git & GitHub
- **Storage:** File-based JSON / structured text
- **Architecture:** Modular, object-oriented, workflow-based design
- **Libraries:** Minimal lightweight utilities (e.g., JSON parsing library)

PROPOSED TIMELINE

Week	Activities
Week 1	Problem identification, requirement analysis
Week 2	Feature planning, project scoping
Week 3	Flowchart and UML class diagram design
Week 4	Proposal submission & presentation
Week 5	Core entity classes implementation
Week 6	Order validation and inventory logic
Week 7	Stock reservation and cancellation handling
Week 8	Invoice generation and payment simulation
Week 9	Shipping workflow and tracking logic
Week 10	Admin dashboard and order management

- Week 11 Data persistence (serialization/deserialization)
- Week 12 Security features (password hashing, admin login)
- Week 13 Logging system and summary report generation
- Week 14 Testing, debugging, documentation
- Week 15 Progress presentation
- Week 16 Final refinement and final presentation

ACTION TAKEN BASED ON FEEDBACK

- Clarified the system as **business-side backend only**
- Removed all customer-facing features
- Expanded admin and reporting capabilities

- Added secure authentication and data persistence
- Improved workflow clarity and failure handling